

CS224 - Spring 2026 - Lab #2 (Version 01: February 12, 18:19)

MIPS Assembly Language Programming: Linked Lists, Recursion Program as Data

Dates: (TAs - Tutor)

Section 1: Thu, 26 Feb, 13:30-17:20 (TA: Kadri, Tarık)

Section 2: Wed, 25 Feb, 13:30-17:20 (TA: Kadri, Mert)

Section 3: Tue, 24 Feb, 13:30-17:20 (TA: Alireza, Mert; Tutor: Berke)

Section 4: Fri, 27 Feb, 08:30-12:20 (TA: Alireza, Tarık)

Section 5: Mon, 23 Feb, 08:30-12:20 (TA: Berkan, Tuna, Yağız)

Section 6: Fri, 27 Feb, 13:30-17:20 (TA: Mahyar, Omid, Tuna)

Lab Location: EA-Z04

TA Full Name (email address: @bilkent.edu.tr) No of Labs

Alireza Dastmalchi Saei: (dastmalchi) 2

Berkan Şahin: (berkan.sahin) 1

Kadri Gofralılar: (kadri.gofralilar) 2

Mahyar Fardinfer (mahyar.fardinfer) 1

Mert Barkın Er (barkin.er) 2

Omid Feizi: (omid.feizi) 2

Pouya Ghahramanin: (ghahramanian) *

Tarık Kaya (tarik.kaya) 2

Tuna Okçu (tuna.okcu) 2

Yağız Özkarahan (yagiz.ozkarahan) 1

* Coordinating TA.

Tutor: Berke Halavurt, berke.halavurt@ug.bilkent.edu.tr

Purpose: This lab aims learning: **1.** Fundamentals of constructing dynamic data structure linked list in assembly language; **2.** Implementation of recursion in assembly language; and **3.** Practicing the Von Neumann's stored program concept.

See the program I in the Moodle assignment folder that show how to create and print a linked list. The same folder also includes another program on accessing the instructions of a program like data. You may use or change these programs according to your needs.

Summary

Preliminary Work: 60 points

1. Generate Summary Linked List (40 points)
2. Program as Data - Instruction Count (20 points).

Lab Work: 40 points

1. Recursive Division (15 points)
2. Recursively Display Linked List in Reverse Order (25 points)

Implementation Notes and Requirements for Subprograms

1. You are not allowed to use \$t registers in the subprograms. In subprograms only use \$s registers to get used to the MIPS software development traditions.
2. For passing arguments to subprograms use argument (\$a) registers and for returning results to the caller use \$v0 and \$v1 registers. When you enter a subprogram save \$s registers that you use in the subprogram to the stack. When necessary, save \$ra register too. If you do not follow this rule 20 points will be taken off from your grade.
3. You can only use a limited number of pseudo instructions in your lab work. They are the following: lw and sw instructions (like: lw \$t0, a; sw \$t0, a), load address (like: la \$t0, array), and conditional branch instructions (like: bgt, blt ...), and load immediate for readability (like: li \$v0, 10).
4. All variable defined in the data segment are at global scope and accessible by all subprograms. However, to use the variables defined in the data segment you have to pass them to subprograms using argument registers. You may freely use ascii messages defined in the data segment without passing them as arguments.

Important Notes for All Labs About Attendance, and Performing-Presenting Your Work
You are obliged to read this document word by word and are responsible for the mistakes you make by not following the rules.

1. Not attending to the lab means 0 out of 100 for that lab. If you attend the lab but do not submit the preliminary part you will lose only the points for the preliminary part. If you attend the lab but do not upload your lab work you will get zero from the lab part.
2. Try to complete the lab part at home. Make sure that you show your work to your TAs and answer their questions to show that you know what you are doing before uploading your lab work and follow the instructions of your TAs.
3. In all labs if you are not told you may assume that inputs are correct. This is for simplicity.
4. In all labs when needed you have to provide a simple user interface for inputs and outputs.
5. Format of your submitted work

You have to provide a neat presentation prepared in txt form. Your programs must be easy to understand and well structured.

Provide following six lines at the top of your submission for preliminary and lab work (make sure that you include the course no. CS224, important for ABET documentation).

CS224

Lab No. (this is Lab No. 2, so enter Lab 2)

Section No.

Your Full Name

Bilkent ID

Date

Please also make sure that your work is identifiable: In terms of which program corresponds to which part of the lab.

- 6. Make sure that the code you submit is really yours and has been internalized. You may experiment with chatGPT-like tools but your work should be yours. You are not allowed to use chatGPT in your final submitted work. Too much help from GenAI tools will disadvantage you in the exams.**

Note that MOSS is capable of detecting ChatGPT code. Furthermore, remember that, the code you use from ChatGPT can also be used by another student in the course.

The purpose of education is learning to learn. Learn how to learn. Use the opportunity provided in this course. It is very probable that in your life you will do things different from what you have learned during your undergraduate education.

DUE DATE FOR PRELIMINARY WORK: SAME FOR ALL SECTIONS

NO late submission will be accepted. Please do not try to break this rule and any other rule we set.

- Please upload your programs of preliminary work to Moodle by 8:30 am on Monday, 23 February, 2026. Please note that the submission closes sharp at 8:30 and no late submissions will be accepted. There can be power or web problems. So do not wait for the last minute.
- You can make resubmissions so do not wait for the last moment. Submit your work earlier and change your submitted work if necessary. Note that only the last submission will be graded.
- Please familiarize yourself with the Moodle course interface, find the submission entry early, and avoid sending an email like "I cannot see the submission interface." The submission interface will be available soon.
- Do not send your work by email attachment: they will not be processed. They have to be in the Moodle system to be processed.
- Use filename **StudentID_FirstName_LastName_SecNo_PRELIM_LabNo.txt** Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero).

DUE DATE FOR LAB WORK: YOUR LAB DAY (different for each section)

- You have to demonstrate your lab work to your TA for grading. Do this by **12:00** in the morning lab and by **17:00** in the afternoon lab. Your TAs may give further instructions on this. If you wait idly and show your work last minute, your work may not be graded.

- b. At the conclusion of the demo for getting your grade, you will **upload your Lab Work** to the Moodle Assignment, for MOSS. See below for the details of lab work submission.
 - c. Try to finish all of your lab work before coming to the lab, but make sure that you upload your work after making sure that it is analyzed by your TA and/or you are given the permission by your TA to upload.
 - d. Do not forget to upload your lab work after it is approved by your TA. If you forget to upload after approval you will get a zero for the lab part of your work.
- 5.

Part 1. Preliminary Work (60 points)

1. Generate Summary Linked List (40 points)

The input to the subprogram is a linked list as defined here: Each node contains Key and Data link fields. Consider the following example input linked list:

--> (15, 20) --> (15, 10) --> (15, 3) --> (20, 40) --> (20, -30) --> (20, -10) --> (55, 17) --> (75, 2) --> (75, 20)

The summary link list generated:

--> (15, 33) --> (55, 17) --> (75, 22)

Note that original input linked list is retained. For example, you may assume that in each node, the first number identifies a company and plays the role of key and the second number indicates the amount of money you will receive from them or you owe to that company. In the example input linked list, for the company number 15, you expect to receive 20, 10 and 3. In the summary linked list, the total is 33, which is the sum of these values associated with this company 15. However, company number 20 does not appear in the summary linked list because $40 - (30 + 10) = 0$.

The subprogram also returns the summary link list size in terms of number of nodes.

The input link list is sorted according to product number and we may have one or more nodes associated with a product number. It can be empty.

Test your program by interactively creating the input linked list. Provide a meaningful user interface in terms for getting and displaying the values.

2. Instruction Count (20 points)

Write a subprogram to find the total number of add and lw instructions used in a program. Return then count for add and lw respectively in \$v0 and \$v1 registers. For example, in the code segment

```
add $t0, $t1, $t2 / sw $t1, 0($t2) / add $t1, $t1, $t2 / lw $t1, 0($t2)
```

there are 2 add instructions and 1 lw instruction.

In MARS Settings SelfModifyingCode option must be turned on. The main program must provide the user interface and it should stop when the user enters a number not within the range of 0 to 31.

Part 2. Lab Work (40 points)

Remember that you are not allowed to use \$t registers in the subprograms: In the subprograms use \$s registers, stack etc.

1. Recursive Division (15 points)

Write a subprogram to perform integer division by successive subtractions. Return the quotient. Assume that the numbers are positive. The user interface provided by the main program should allow user to try it any number of times. When one of the numbers is zero it stops.

2. Display Linked List in Reverse Order Recursively (25 points)

Write a recursive subprogram to print a linked list in reverse order. As input use the linked list structure defined above in Preliminary Work Program 1.

For example, the linked list --> (15, 33) --> (20, 40) --> (55, 17) --> (75, 22)
will be displayed as --> (75, 22) --> (55, 17) --> (20, 40) --> (15, 33)

Part 3. Submit Lab Work for MOSS Similarity Testing

1. Submit your Lab Work MIPS codes for MOSS similarity- plagiarism testing to Moodle.
2. You will upload one file. Use filename **StudentID_FirstName_LastName_SecNo_LAB_LabNo.txt**
3. Only a NOTEPAD FILE (txt file) is accepted. No txt file upload means you get 0 from the lab. Please note that we have several students and efficiency is important.
4. *Even if you didn't finish, or didn't get the MIPS codes working, you must submit your code to the Moodle Assignment for similarity checking.*

Part 4. Cleanup

1. After saving any files that you might want to have in the future to your own storage device, erase all the files you created from the computer in the lab.
2. When applicable put back all the hardware, boards, wires, tools, etc where they came from.
3. Clean up your lab desk, to leave it completely clean and ready for the next group who will come.

LAB POLICIES

1. You can do the lab only in your section. Missing your section time and doing in another day is not allowed.
2. The questions asked by the TA will have an effect on your lab score.
3. Lab score will be reduced to 0 if the code is not submitted for similarity testing, or if it is plagiarized. MOSS-testing will be done, to determine similarity rates. Trivial changes to code will not hide plagiarism from MOSS—the algorithm is quite sophisticated and powerful works for ChatGPT code too. Please also note that obviously you should not use any program available on the web, or in a book, etc. since MOSS will find it. The use of the ideas we discussed in the classroom is not a problem.

4. You must be in lab, working on the lab, from the time lab starts until your work is finished and you leave.
5. No cell phone usage during lab.
6. Internet usage is permitted only to lab-related technical sites.